

Classes and Objects

Object-Oriented Programming (OOP): A programming paradigm that involves designing programs around concepts represented as "objects"

- Python supports OOP through the provision of classes.
- Terminology
 - Class: A collection of functions and attributes, attached to a specific name, which represents an abstract concept.
 - Attribute: A named piece of data (i.e. variable associated with a class.
 - Object: A single concrete instance generated from a class

Instances of Classes

Classes can be viewed as factories or templates for generating new object instances.

Each object instance takes on the properties of the class from which it was created.

Instances of Classes

Abstract
concept



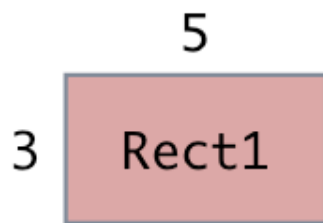
Attributes

- width
- height
- colour

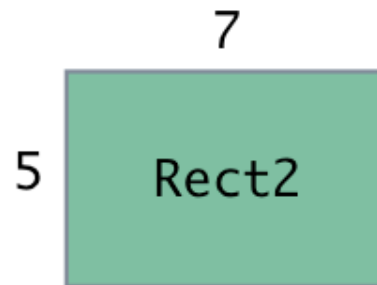
Functions

- area()

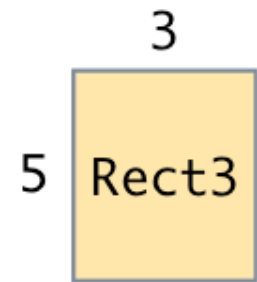
Multiple
concrete
instances



width=5
height=3
colour=(200,0,0)
area() -> 15



width=7
height=5
colour=(0,200,0)
area() -> 35



width=3
height=5
colour=(255,200,0)
area() -> 15

Creating Classes

Defining a class in Python is done using the class keyword, followed by an indented block with the class contents.

Class data
attributes



```
class <Classname>:
    data1 = value1
    ...
    dataM = valueM

    def <function1>(self, arg1, ..., argK):
        <block>
    ...
    def <functionN>(self, arg1, ..., argK):
        <block>
```

Class
member
functions



Defining Functions in Classes

- A class definition block can include multiple functions.
- These represent the functionality or behaviors that are associated with the class.

```
>>> class Maths:  
...     def subtract(self,i,j):  
...         return i-j  
...  
...     def add(self,x,y):  
...         return x+y
```

Argument (self) refers to the object itself

Calling Functions in Classes

- Using Class Functions from Outside a Class
Functions are referenced by using the dot syntax:
<objectName>.<methodName>()

```
>>> m = Maths()  
>>> m.subtract(10,5)  
5  
>>> m.add(6,7)  
13
```

← No need to
specify value for
self, Python does
this automatically

Calling Functions in Classes

- Using Class Functions from Inside a Class

When referring to functions from within a class, we must always prefix the function name with self

(e.g. self.subtract())

```
>>> class Maths:
...     def subtract(self,i,j):
...         return i-j
...
...     def testsub(self):
...         print self.subtract(8,4)
```

Tell Python to use function
associated with this object



Attributes

Class attribute
defined at top of
class

```
>>> class Person:
...     company = "ucd"
...
...     def __init__(self):
...         self.age = 23
```

Instance attribute
defined inside a class
function.
The **self** prefix is
always required.

```
>>> p1 = Person()
>>> p2 = Person()
>>> p1.age = 35
>>> print p2.age
23
```

Change to instance attribute age
affects only the associated
instance (p2)

```
>>> p1 = Person()
>>> p2 = Person()
>>> p1.company = "ibm"
>>> print p2.company
'ibm'
```

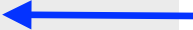
Change to class attribute company
affects all instances (p1 and p2)

Constructor

- When an instance of a class is created, the class constructor function is automatically called.
- The constructor is always named `__init__()`
- It contains code for initializing a new instance of the class to a specific initial state (e.g. setting instance attribute values).

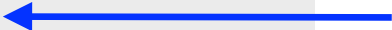
```
>>> class Person:
...     def __init__( self, s ):
...         self.name = s
...
...     def hello( self ):
...         print "Hello", self.name
```

Constructor function taking
initial value for instance
attribute name



```
>>> t = Person("John")
>>> t.hello()
Hello John
```

Calls `__init__()`
on Person



Inheritance

Class inheritance is designed to model relationships of the type "x is a y" (e.g. "a triangle is a shape")

